

eleVADR

Database Architecture and MongoDB Schema

Technical reference for the Windows platform deployment

Scope. This document describes the MongoDB data model used by the current eleVADR platform architecture: account identity, report metadata, retained-capture metadata, saved Detection Context profiles, and audit events. It also explains the filesystem relationship for large PCAP and report objects.

Collection	Primary purpose	Bulk data stored in MongoDB?
users	Local user identities, roles, password hashes, account state	No bulk evidence
reports	Saved-report catalog, ownership, counts, filesystem reference	No - complete report JSON is filesystem-backed
captures	Retained-PCAP catalog, hash, size, retention metadata	No - PCAP/PCAPNG is filesystem-backed
context_profiles	Owner-scoped Detection Context configuration	Yes - profile document
audit_events	Security and platform activity trail	Yes - event metadata

Architecture at a glance

MongoDB: identity + ownership + indexes + metadata + configuration + audit
Filesystem: large PCAP evidence + complete JSON reports

```
users._id ---> reports.owner_id
+---> captures.owner_id
+---> context_profiles.owner_id
+---> audit_events.actor_id

reports.capture_id ---> captures.capture_id
```

1. Storage architecture

eleVADR uses a hybrid persistence model. MongoDB provides queryable platform records and ownership boundaries; large analysis artifacts remain filesystem-backed. This avoids placing potentially large packet captures and complete report payloads into ordinary MongoDB documents while still allowing the application to search, authorize, sort, retain, and audit them.

Filesystem mapping

```
storage/  
users/  
<owner-id>/  
reports/  
<report-id>.json  
pcaps/  
<capture-id>.pcap (or .pcapng)
```

Operational consequence

A complete backup or restore must treat MongoDB and the filesystem storage tree as one logical dataset. A MongoDB record without its referenced file is incomplete; a file without its corresponding catalog record is an orphan candidate.

Ownership model

Report, capture, and Context records are owner-scoped. Application authorization should use the authenticated user identifier rather than trusting a username supplied by the client. This is also why compound indexes include owner identifiers.

Schema conventions

Convention	Meaning
owner_id	Stable account identifier used to scope user-owned objects
owner_username	Human-readable username copied for operational/audit convenience
created_at / updated_at	UTC timestamps maintained by the platform
report_id / capture_id / profile_id	Application-level identifiers independent of MongoDB _id
path fields	Filesystem references; not the bulk artifact itself

2. users collection

Authentication, authorization, and account lifecycle.

Fields

Field	BSON/type	Req.	Purpose
_id	ObjectId	Yes	MongoDB document identifier
username	String	Yes	Display/login username
username_normalized	String	Yes	Canonical value used for uniqueness/login lookup
password_hash	String	Yes	Password verifier; never plaintext
role	String	Yes	admin, analyst, or read_only
disabled	Boolean	Yes	Prevents account use when true
email	String	No	Optional email, if configured
email_normalized	String	No	Canonical optional email
created	Date	Yes	Account creation time
updated	Date	Yes	Last account update
last_login	Date	No	Most recent successful login

Indexes and constraints

Index	Keys	Behavior
users_username_unique	username_normalized	Unique
users_email_unique	email_normalized	Unique, partial for string values

Representative document

```
{
  "username": "bryan",
  "username_normalized": "bryan",
  "password_hash": "<password verifier>",
  "role": "admin",
  "disabled": false,
  "created": ISODate("..."),
  "updated": ISODate("..."),
  "last_login": ISODate("...")
}
```

Security note

The email uniqueness rule should be implemented as a partial index so accounts without an email do not collide on a shared null value. Password hashes are sensitive authentication data and should not appear in logs, exports, or audit metadata.

3. reports collection

Catalog for saved reports. The complete report JSON remains on disk.

Fields

Field	BSON/type	Req.	Purpose
_id	ObjectId	Yes	MongoDB document identifier
owner_id	String	Yes	Owning user identifier
owner_username	String	Yes	Owning username snapshot
report_id	String	Yes	Application report identifier
title	String	Yes	User-facing report title
source_filename	String	Yes	Original PCAP/report source name
report_path	String	Yes	Filesystem path/reference to JSON
report_version	String	Yes	eleVADR report format version
finding_count	Integer	Yes	Cached finding count for listings
device_count	Integer	Yes	Cached device count for listings
capture_id	String	No	Links report to retained capture when applicable
created_at	Date	Yes	Saved time
updated_at	Date	Yes	Last metadata update

Indexes and constraints

Index	Keys	Behavior
reports_owner_created	owner_id + created_at DESC	Query/sort
reports_owner_report_unique	owner_id + report_id	Unique

Representative document

```
{
  "owner_id": "<user-id>",
  "owner_username": "bryan",
  "report_id": "elevadr-...",
  "title": "01_mixed_ot_baseline.pcap",
  "source_filename": "01_mixed_ot_baseline.pcap",
  "report_path": "users/<owner>/reports/<report>.json",
  "report_version": "2.x",
  "finding_count": 4,
  "device_count": 12,
  "capture_id": "<capture-id>",
  "created_at": ISODate("..."),
  "updated_at": ISODate("...")
}
```

4. captures collection

Catalog and retention metadata for retained packet captures.

Fields

Field	BSON/type	Req.	Purpose
_id	ObjectId	Yes	MongoDB document identifier
capture_id	String	Yes	Application capture identifier
owner_id	String	Yes	Owning user identifier
owner_username	String	Yes	Owning username snapshot
filename	String	Yes	Original filename
sha256	String	Yes	SHA-256 content digest
size_bytes	Integer	Yes	Stored capture size
capture_path	String	Yes	Filesystem path/reference
created_at	Date	Yes	Retention creation time
last_used_at	Date	Yes	Most recent use/re-analysis time

Indexes and constraints

Index	Keys	Behavior
captures_owner_created	owner_id + created_at DESC	Query/sort
captures_owner_sha_unique	owner_id + sha256	Unique
captures_last_used	last_used_at	Retention/cleanup query

Representative document

```
{
  "capture_id": "<uuid>",
  "owner_id": "<user-id>",
  "owner_username": "bryan",
  "filename": "01_mixed_ot_baseline.pcap",
  "sha256": "<64 hex chars>",
  "size_bytes": 12345678,
  "capture_path": "users/<owner>/pcaps/<capture>.pcap",
  "created_at": ISODate("..."),
  "last_used_at": ISODate("...")
}
```

Deduplication behavior

The owner + SHA-256 uniqueness constraint means identical capture content is deduplicated per owner rather than globally across users. This preserves tenant isolation while preventing redundant retained copies for the same account.

5. context_profiles collection

Saved, owner-scoped Detection Context configurations.

Fields

Field	BSON/type	Req.	Purpose
_id	ObjectId	Yes	MongoDB document identifier
owner_id	String	Yes	Owning user identifier
owner_username	String	Yes	Owning username snapshot
profile_id	String	Yes	Application profile identifier
profile	Document	Yes	Complete nested Detection Context profile
created_at	Date	Yes	Creation time
updated_at	Date	Yes	Last profile save

Indexes and constraints

Index	Keys	Behavior
context_profiles_owner_id	owner_id + profile_id	Unique
context_profiles_owner_updated	owner_id + updated_at DESC	Query/sort

Representative document

```
{
  "owner_id": "<user-id>",
  "owner_username": "bryan",
  "profile_id": "<profile-id>",
  "profile": {
    "id": "<profile-id>",
    "name": "Plant Network",
    "updatedAt": "...",
    "...": "complete Detection Context structure"
  },
  "created_at": ISODate("..."),
  "updated_at": ISODate("...")
}
```

Document-model rationale

Detection Context is stored as one nested profile document rather than decomposed into many relational-style collections. The application normally loads and saves the Context as a coherent configuration object, which fits MongoDB's document model.

6. audit_events collection

Append-oriented security and platform activity history.

Fields

Field	BSON/type	Req.	Purpose
_id	ObjectId	Yes	MongoDB document identifier
created_at	Date	Yes	Event time
actor_id	String	No	Authenticated actor; may be absent for system events
actor_username	String	No	Human-readable actor
action	String	Yes	Operation name, e.g. capture.reanalyze
result	String	Yes	success, failure, or denied
target_type	String	No	Object category such as capture/user/storage
target_id	String	No	Target identifier
metadata	Document	No	Sanitized action-specific details

Indexes and constraints

Index	Keys	Behavior
audit_created_desc	created_at DESC	Recent activity
audit_actor_created	actor_id + created_at DESC	Per-user activity
audit_retention_ttl	created_at	TTL retention

Representative document

```
{
  "created_at": ISODate("..."),
  "actor_id": "<user-id>",
  "actor_username": "bryan",
  "action": "capture.reanalyze",
  "result": "success",
  "target_type": "capture",
  "target_id": "<capture-id>",
  "metadata": { "jobId": "..." }
}
```

Retention and sanitization

The platform uses a TTL index for audit retention (the current documented default is 180 days). Audit metadata should redact password-, token-, secret-, authorization-, cookie-, credential-, and raw-PCAP-like values before insertion.

7. Relationships and lifecycle

```
users._id
|
+-- reports.owner_id ---- report_path ----> JSON report file
| |
| +-- capture_id -----+
| |
+-- captures.owner_id <-----+
| |
| +-- capture_path --> PCAP/PCAPNG file
|
+-- context_profiles.owner_id
|
+-- audit_events.actor_id
```

Report creation

A PCAP analysis can produce both a retained capture record and a saved report record. The report may retain `capture_id` so later analysis history and comparison can group multiple reports produced from the same capture.

Re-analysis

Re-analysis uses the retained capture and current Detection Context/module selection. A new report can be created while the original capture remains the same, allowing analysis history to compare findings and configuration changes over time.

Deletion and cleanup

Deleting a catalog record and deleting its filesystem object are distinct operations and should be coordinated. Storage cleanup also checks retention policy and orphan files. Administrative backup/restore should preserve MongoDB and filesystem consistency.

Authorization boundary

Owner-scoped queries should include `owner_id`. Cross-user administrative views are explicit privileged operations rather than the default data-access path.

8. Administrator inspection examples

The following examples are intended for an authorized administrator using mongosh against the local eleVADR database. Exact connection parameters come from the platform environment configuration.

List collections and indexes

```
use elevadr
show collections

db.users.getIndexes()
db.reports.getIndexes()
db.captures.getIndexes()
db.context_profiles.getIndexes()
db.audit_events.getIndexes()
```

Inspect recent records without exposing password hashes

```
db.users.find({}, {password_hash: 0}).sort({created: -1})

db.reports.find({owner_username: "bryan"}).sort({created_at: -1}).limit(20)

db.captures.find({owner_username: "bryan"}).sort({created_at: -1}).limit(20)

db.audit_events.find({actor_username: "bryan"}).sort({created_at: -1}).limit(50)
```

Storage-oriented aggregation

```
db.captures.aggregate([
  {$group: {_id: "$owner_id", captures: {$sum: 1}, bytes: {$sum: "$size_bytes"}}},
  {$sort: {bytes: -1}}
])
```

Safety note

Avoid direct database edits during normal operation. Use eleVADR application/API workflows for account, report, capture, Context, cleanup, and retention operations so filesystem changes, authorization checks, and audit events remain consistent.

9. Schema governance and backup considerations

MongoDB is schemaless at the storage engine level, but eleVADR still relies on an application schema. Field names, types, indexes, ownership semantics, and filesystem references are therefore part of the platform contract.

Recommended controls

Control	Reason
Index creation at startup/migration	Guarantees uniqueness and query behavior before serving requests
Application-level validation	Prevents malformed roles, identifiers, paths, and nested Context data
Owner-scoped access checks	Prevents cross-account report/capture/profile disclosure
Filesystem path confinement	Prevents path traversal outside the configured storage root
Backup both persistence layers	MongoDB alone is not a complete eleVADR backup
Restore validation	Detects missing files, orphan files, broken ownership references, and incompatible schema state
Audit redaction	Prevents credentials and sensitive raw evidence from entering the event log

Backup set

Logical eleVADR backup =
MongoDB database state
+ filesystem report storage
+ filesystem retained-PCAP storage
+ required platform configuration/secrets handled according to deployment policy

Versioning note

This reference describes the current platform data model discussed and validated during the eleVADR Windows platform work. If collection definitions or indexes change, update this document together with the migration/deployment tests so operational documentation does not drift from the code.